SME DataPrep examples

Aimar Barrena

Introduction

This document illustrates the main functions of the `smeDataPrep` package with two supervised tabular datasets. The first dataset is used as a general walkthrough of the available tools. The second dataset is used for a shorter, more realistic analysis where we select only the most relevant functions and plots for that dataset.

Example 1: Example use of all the available functions in the package

The goal of this section is not to study a specific historical dataset. The dataset is simply a convenient example of a supervised table with numerical variables, categorical variables, missing values, and a binary target. This is the kind of structure where the package functions are useful.

1. Load and select variables

We keep a small set of columns so that the outputs are easy to interpret. The target is binary, and the explanatory variables include both numerical and categorical attributes.

```
VARIABLES <- c(
  "Survived",
  "Pclass",
  "Sex",
  "Age",
  "SibSp",
  "Parch",
  "Fare",
  "Embarked"
)
TARGET <- "Survived"

dataset <- read_sme_dataset(
  "data/titanic/train.csv",
  target = TARGET,
  variables = VARIABLES
)
head(dataset$data)
```

```
##   Survived Pclass    Sex Age SibSp Parch    Fare Embarked
## 1        0      3   male  22     1     0  7.2500        S
## 2        1      1 female  38     1     0 71.2833        C
## 3        1      3 female  26     0     0  7.9250        S
## 4        1      1 female  35     1     0 53.1000        S
## 5        0      3   male  35     0     0  8.0500        S
## 6        0      3   male  NA     0     0  8.4583        Q
```

2. Inspect the dataset before transforming it

Before calculating metrics or changing variables, we check whether the target is valid, what type of variables we have, and where missing values appear.

```
validate_binary_target(dataset$data, TARGET)
```

```
## [1] TRUE
```

```
detect_variable_types(dataset$data)
```

```
##   variable   r_class detected_type missing_count missing_rate unique_count
## 1 Survived   integer        binary             0    0.0000000            2
## 2   Pclass   integer     numerical             0    0.0000000            3
## 3      Sex character        binary             0    0.0000000            2
## 4      Age   numeric     numerical           177    0.1986532           88
## 5    SibSp   integer     numerical             0    0.0000000            7
## 6    Parch   integer     numerical             0    0.0000000            7
## 7     Fare   numeric     numerical             0    0.0000000          248
## 8 Embarked character   categorical             0    0.0000000            4
##   unique_rate
## 1 0.002244669
## 2 0.003367003
## 3 0.002244669
## 4 0.123249300
## 5 0.007856341
## 6 0.007856341
## 7 0.278338945
## 8 0.004489338
```

The type report separates the target from numerical and categorical variables. `Survived` is detected as binary, `Age` and `Fare` are numerical, and variables such as `Sex` and `Embarked` are categorical. This tells us which functions are appropriate for each column.

```
missing_value_report(dataset$data)
```

```
##          variable missing_count missing_rate
## Age           Age           177    0.1986532
## Survived Survived             0    0.0000000
## Pclass     Pclass             0    0.0000000
## Sex           Sex             0    0.0000000
## SibSp       SibSp             0    0.0000000
## Parch       Parch             0    0.0000000
## Fare         Fare             0    0.0000000
## Embarked Embarked             0    0.0000000
```

The missing-value report shows that `Age` has 177 missing values, about 19.9% of the selected rows. `Embarked` has only 2 missing values. This is a useful preprocessing finding: some functions can ignore missing values, but for a clean analysis dataset we may want to impute them.

```
imputed_data <- impute_missing_values(dataset$data, strategy = "auto")
missing_value_report(imputed_data)
```

```
##          variable missing_count missing_rate
## Survived Survived             0            0
## Pclass     Pclass             0            0
## Sex           Sex             0            0
## Age           Age             0            0
```

```
## SibSp       SibSp             0            0
## Parch       Parch             0            0
## Fare         Fare             0            0
## Embarked Embarked             0            0
```

After automatic imputation, missing values disappear from this selected dataset. The automatic rule uses the median for numerical variables and the most frequent value for categorical variables.

3. Calculate metrics by variable type

`attribute_metrics` chooses the metric according to the variable type: variance and AUC for numerical variables, entropy for categorical variables.

```
metrics <- attribute_metrics(dataset$data, target = TARGET, positive_class = 1)
metrics
```

```
##    attribute   metric        value
## 1     Pclass variance    0.6982306
## 2     Pclass      auc    0.3185830
## 3        Sex  entropy    0.9362046
## 4        Age variance  210.7235798
## 5        Age      auc    0.4691119
## 6      SibSp variance    1.2146783
## 7      SibSp      auc    0.5431619
## 8      Parch variance    0.6489990
## 9      Parch      auc    0.5612171
## 10      Fare variance 2466.6653117
## 11      Fare      auc    0.6921223
## 12  Embarked  entropy    1.1173935
```

How to read this table: variance measures spread, entropy measures diversity in categories, and AUC measures how well a numerical variable orders the two target classes. In this output, `Fare` has a higher AUC than `Age`, meaning fare is more informative than age as a single numerical ranking variable in this example. `Age` has an AUC below 0.5, so larger values of age are not associated with the positive class under this direct ordering.

4. Discretize numerical variables

Discretization converts numerical values into bins. Equal-width bins split the numerical range into intervals of the same size. Equal-frequency bins try to place a similar number of rows in each bin.

```
numeric_features <- c("Age", "SibSp", "Parch", "Fare")

head(discretize_dataset_equal_width(dataset$data[numeric_features], bins = 4))
```

```
##     Age SibSp Parch  Fare
## 1 bin_2 bin_1 bin_1 bin_1
## 2 bin_2 bin_1 bin_1 bin_1
## 3 bin_2 bin_1 bin_1 bin_1
## 4 bin_2 bin_1 bin_1 bin_1
## 5 bin_2 bin_1 bin_1 bin_1
## 6  <NA> bin_1 bin_1 bin_1
```

```
head(discretize_dataset_equal_frequency(dataset$data[numeric_features], bins = 4))
```

```
##     Age SibSp Parch  Fare
## 1 bin_2 bin_3 bin_1 bin_1
## 2 bin_3 bin_3 bin_1 bin_4
## 3 bin_2 bin_1 bin_1 bin_2
```

```
## 4 bin_3 bin_3 bin_1 bin_4
## 5 bin_3 bin_1 bin_1 bin_2
## 6  <NA> bin_1 bin_1 bin_2
```

Equal-width bins are easier to interpret because each interval has the same numerical size. Equal-frequency bins are often more balanced because they adapt to the empirical distribution of the variable.

```
age_groups <- discretize_by_thresholds(
  dataset$data$Age,
  thresholds = c(12, 60),
  labels = c("child", "adult", "older")
)

fare_sd_groups <- discretize_by_standard_deviation(dataset$data$Fare)

head(data.frame(
  Age = dataset$data$Age,
  age_group = age_groups,
  Fare = dataset$data$Fare,
  fare_sd_group = fare_sd_groups
), 10)
```

```
##    Age age_group    Fare fare_sd_group
## 1   22     adult  7.2500       typical
## 2   38     adult 71.2833       typical
## 3   26     adult  7.9250       typical
## 4   35     adult 53.1000       typical
## 5   35     adult  8.0500       typical
## 6   NA      <NA>  8.4583       typical
## 7   54     adult 51.8625       typical
## 8    2     child 21.0750       typical
## 9   27     adult 11.1333       typical
## 10  14     adult 30.0708       typical
```

Manual thresholds are useful when the cut points have a real meaning. Standard-deviation discretization is different: it classifies values according to their distance from the mean, so the groups describe relative position rather than fixed domain thresholds.

5. Normalize, standardize, filter, and compare associations

```
head(normalize_dataset(dataset$data[numeric_features]))
```

```
##         Age SibSp Parch       Fare
## 1 0.2711737 0.125     0 0.01415106
## 2 0.4722292 0.125     0 0.13913574
## 3 0.3214375 0.000     0 0.01546857
## 4 0.4345313 0.125     0 0.10364430
## 5 0.4345313 0.000     0 0.01571255
## 6        NA 0.000     0 0.01650950
```

```
head(standardize_dataset(dataset$data[numeric_features]))
```

```
##          Age      SibSp      Parch       Fare
## 1 -0.5303766  0.4327934 -0.4736736 -0.5024452
## 2  0.5718310  0.4327934 -0.4736736  0.7868453
## 3 -0.2548247 -0.4745452 -0.4736736 -0.4888543
```

```
## 4  0.3651671  0.4327934 -0.4736736  0.4207302
## 5  0.3651671 -0.4745452 -0.4736736 -0.4863374
## 6         NA -0.4745452 -0.4736736 -0.4781164
```

Normalized values are between 0 and 1. Standardized values are centered around 0 and expressed in standard deviations. These transformations are useful when variables have different units or scales.

```
filtered_by_auc <- filter_variables(
  dataset$data,
  metric = "auc",
  threshold = 0.55,
  target = TARGET,
  positive_class = 1
)
head(filtered_by_auc)
```

```
##   Parch    Fare Survived
## 1     0  7.2500        0
## 2     0 71.2833        1
## 3     0  7.9250        1
## 4     0 53.1000        1
## 5     0  8.0500        0
## 6     0  8.4583        0
```

The filtered dataset keeps only variables that pass the selected metric threshold. With an AUC threshold of 0.55, the retained variables are the ones that show at least modest separation between the two classes.

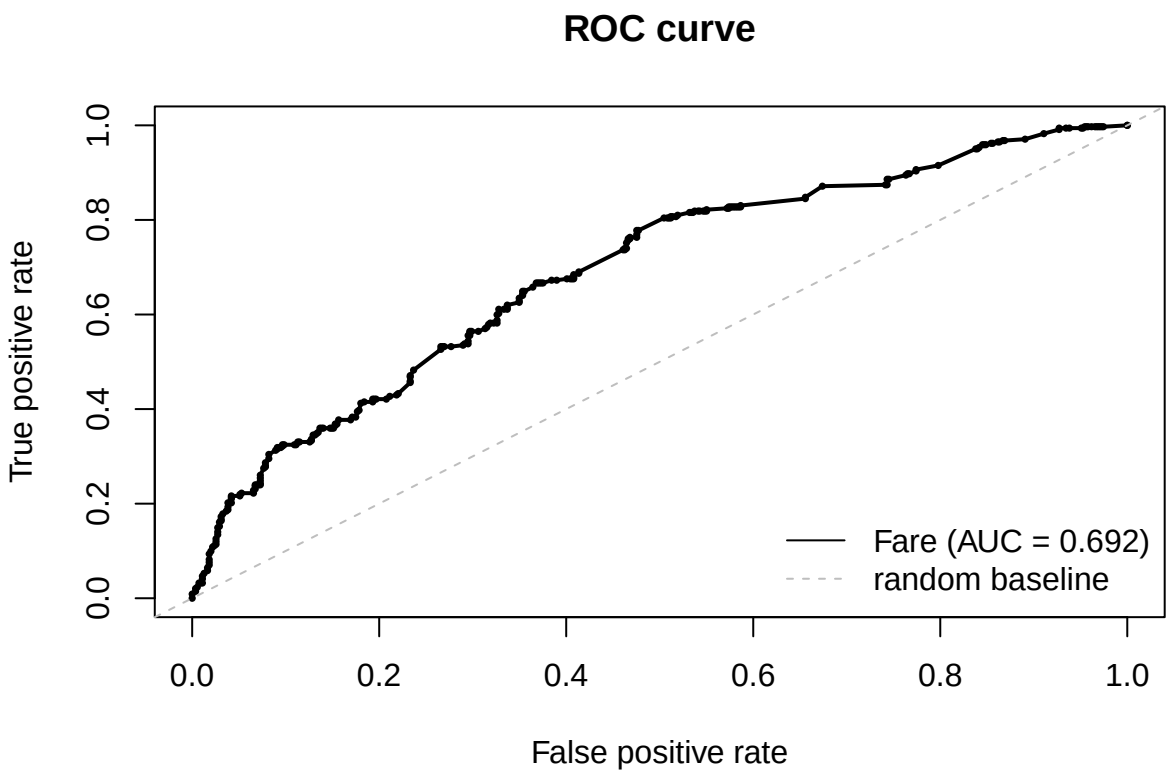
```
matrix <- association_matrix(dataset$data[c("Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"
matrix
```

```
##               Pclass        Sex         Age       SibSp       Parch        Fare
## Pclass    1.00000000         NA -0.36922602  0.08308136  0.01844267 -0.54949962
## Sex               NA 0.93620464          NA          NA          NA          NA
## Age       -0.36922602         NA  1.00000000 -0.30824676 -0.18911926  0.09606669
## SibSp      0.08308136         NA -0.30824676  1.00000000  0.41483770  0.15965104
## Parch      0.01844267         NA -0.18911926  0.41483770  1.00000000  0.21622494
## Fare      -0.54949962         NA  0.09606669  0.15965104  0.21622494  1.00000000
## Embarked          NA 0.01397704          NA          NA          NA          NA
##            Embarked
## Pclass           NA
## Sex      0.01397704
## Age              NA
## SibSp            NA
## Parch            NA
## Fare             NA
## Embarked 1.11739345
```

The association matrix uses Pearson correlation for numerical pairs and mutual information for categorical pairs. Mixed numerical-categorical pairs appear as missing values in this version because they require a different association measure.

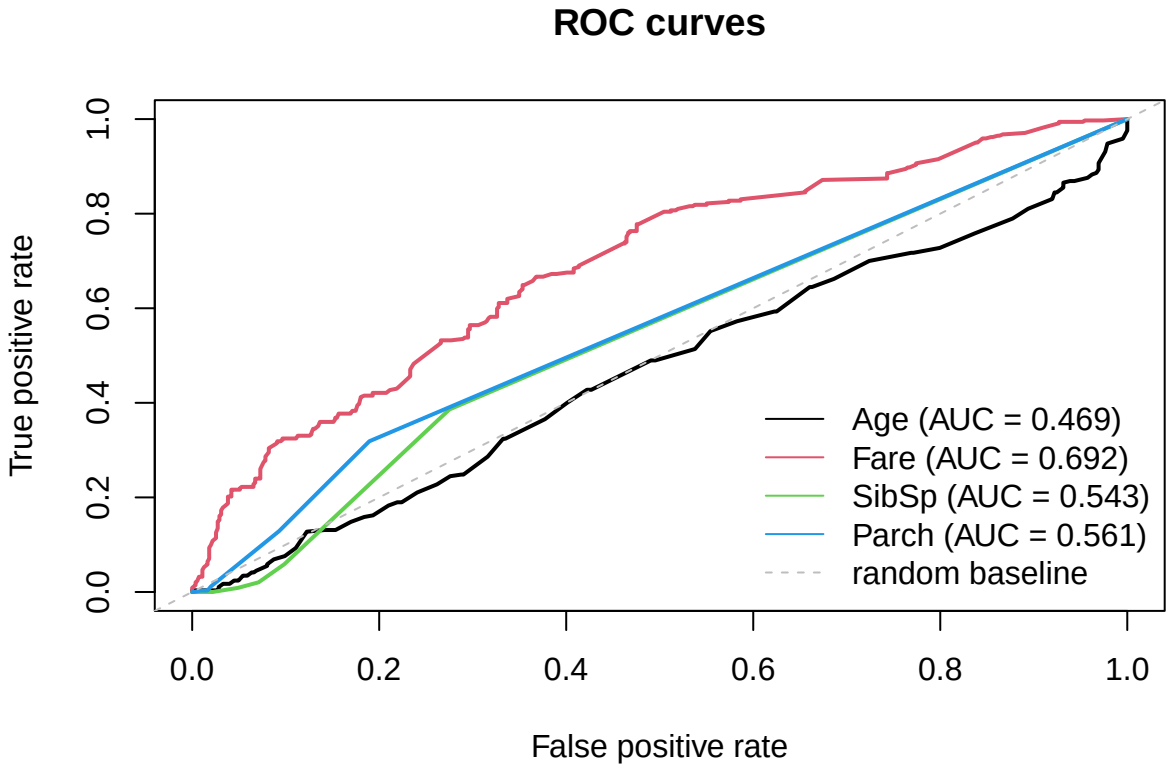
6. Interpret the plots

```
plot_roc_curve(dataset$data$Fare, dataset$data$Survived, positive_class = 1, label = "Fare")
```

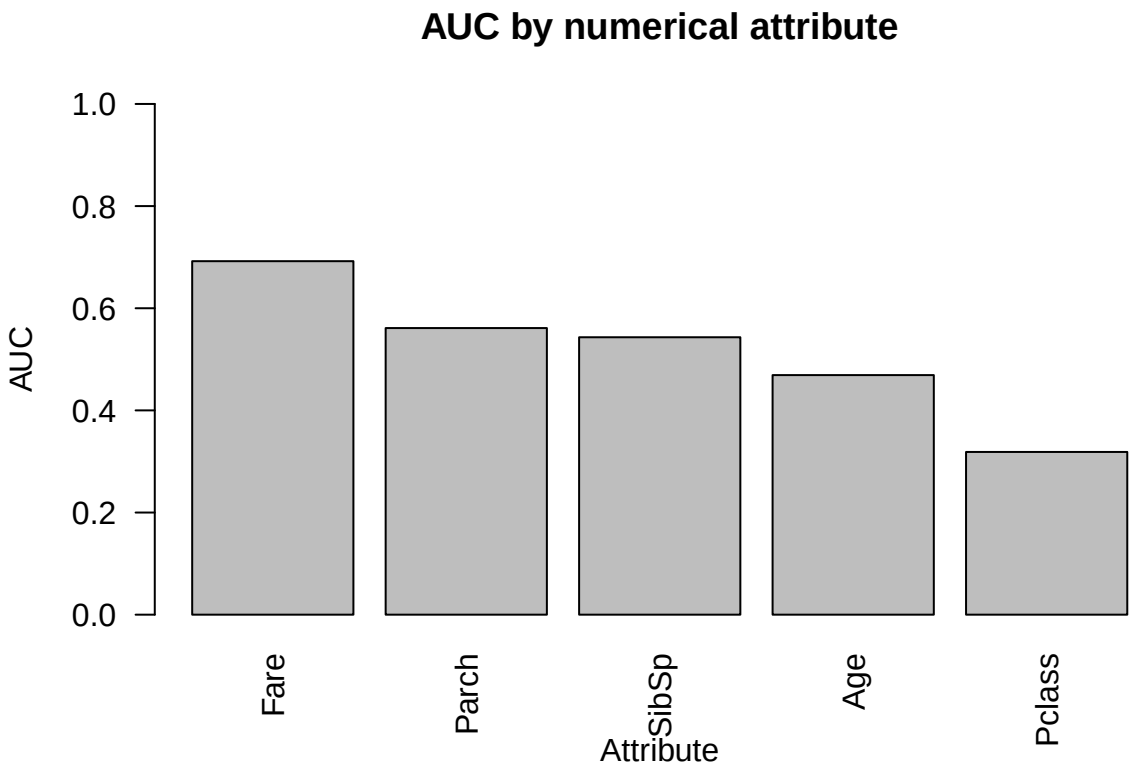


The ROC curve shows the tradeoff between true positive rate and false positive rate for all thresholds of `Fare`. A curve close to the diagonal would mean random ordering. Here the curve is above the diagonal, which matches the AUC value above 0.5.

```
plot_roc_curves(dataset$data, target = TARGET, columns = c("Age", "Fare", "SibSp", "Parch"), positive_cl
```

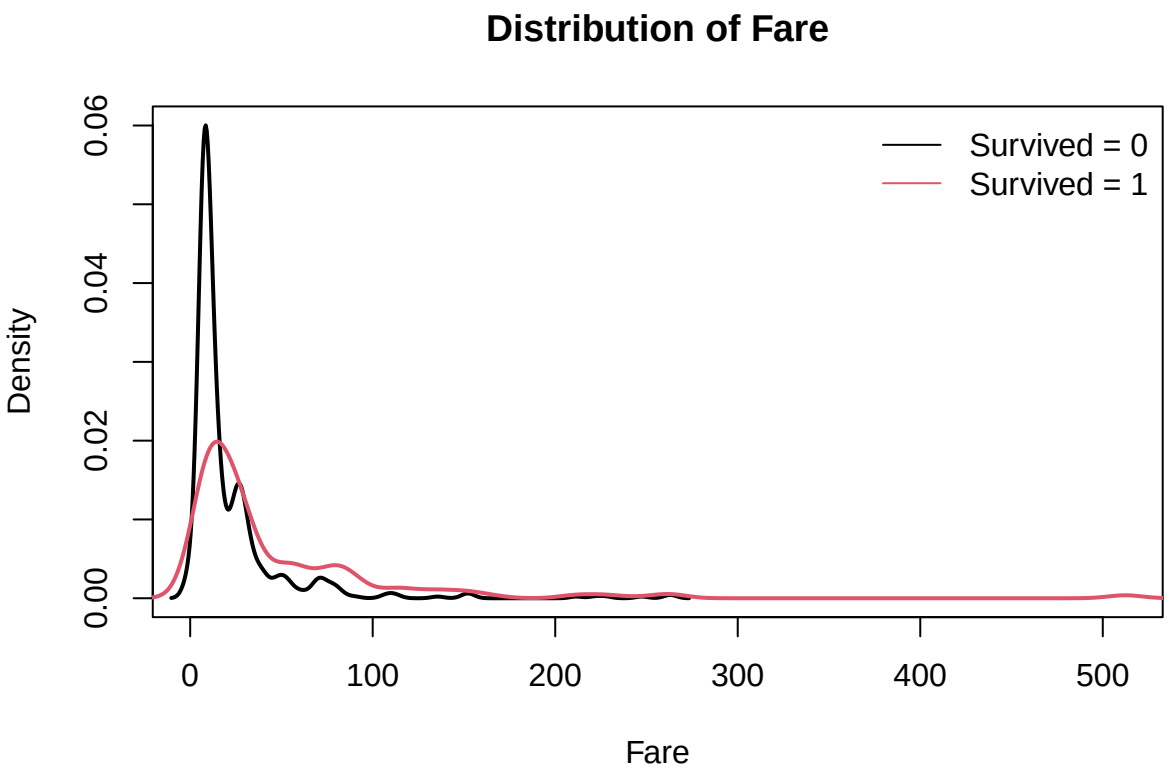


```r
compare_auc_values(dataset$data, target = TARGET, positive_class = 1)
```

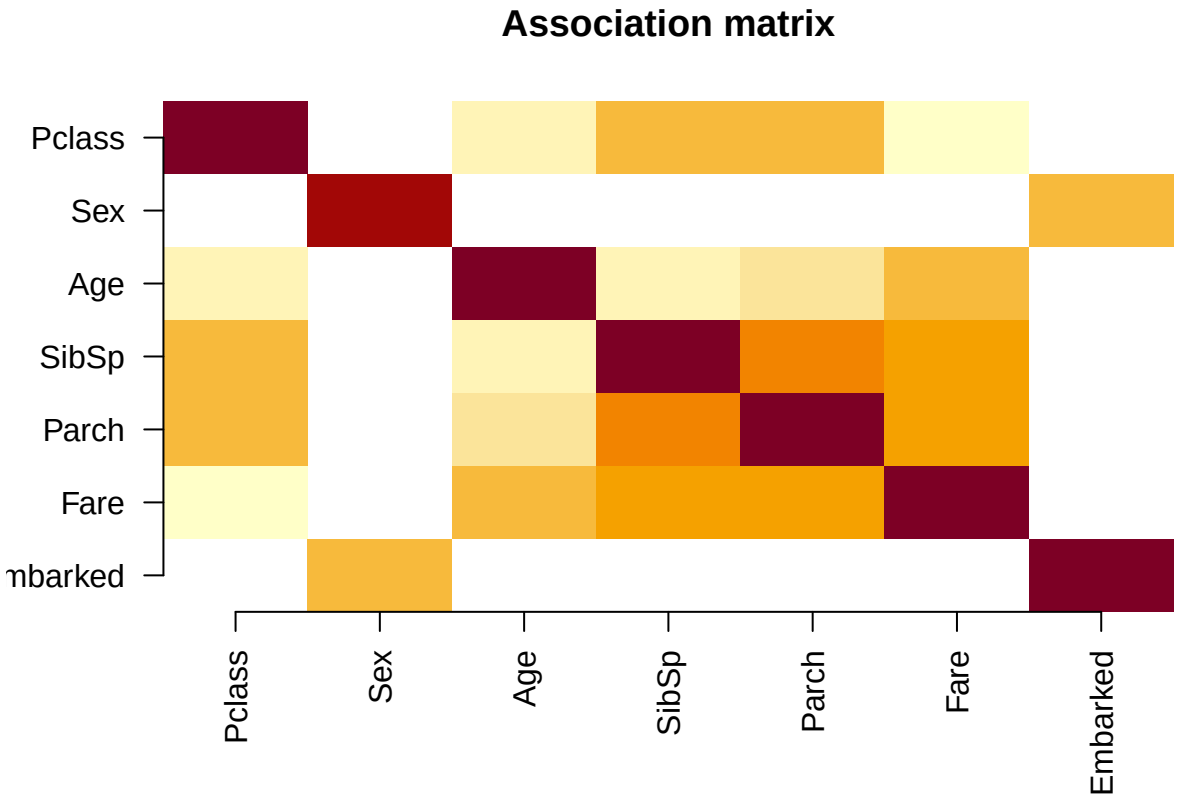


The multi-ROC plot and the AUC comparison barplot answer related questions. The ROC plot shows the full threshold behavior of each variable, while the barplot gives a compact ranking by final AUC.

```r
plot_variable_distribution(dataset$data, variable = "Fare", target = TARGET, by_class = TRUE)
```




```
plot_association_matrix(matrix)
```



The distribution plot helps compare the shape of one numerical variable across target classes. The matrix plot makes pairwise relationships easy to scan visually.

Example 2: Real-world example

This final section uses a health dataset. Instead of using every function again, we apply a smaller set of relevant tools: target validation, missing-value/type inspection, descriptive comparison by class, AUC metrics, ROC curves, and distribution plots.

This part also shows how we can directly use some of these functions with the dataset object.

```
diabetes_data <- read_sme_dataset(
  "data/diabetes/diabetes_prediction_dataset.csv",
  target = "diabetes"
)
head(diabetes_data$data)
```

```
##   gender age hypertension heart_disease smoking_history   bmi HbA1c_level
## 1 Female  80            0             1           never 25.19         6.6
## 2 Female  54            0             0         No Info 27.32         6.6
## 3   Male  28            0             0           never 27.32         5.7
## 4 Female  36            0             0         current 23.45         5.0
## 5   Male  76            1             1         current 20.14         4.8
## 6 Female  20            0             0           never 27.32         6.6
##   blood_glucose_level diabetes
## 1                 140        0
## 2                  80        0
## 3                 158        0
## 4                 155        0
## 5                 155        0
## 6                  85        0
```

```
summary_sme_dataset(diabetes_data)
```

```
## $n_rows
## [1] 100000
##
## $n_columns
## [1] 9
##
## $variables
##              variable   r_class detected_type missing_count missing_rate
## 1              gender character   categorical             0            0
## 2                 age   numeric     numerical             0            0
## 3        hypertension   integer        binary             0            0
## 4       heart_disease   integer        binary             0            0
## 5     smoking_history character   categorical             0            0
## 6                 bmi   numeric     numerical             0            0
## 7         HbA1c_level   numeric     numerical             0            0
## 8 blood_glucose_level   integer     numerical             0            0
## 9            diabetes   integer        binary             0            0
##   unique_count unique_rate
## 1            3     0.00003
## 2          102     0.00102
## 3            2     0.00002
## 4            2     0.00002
## 5            6     0.00006
## 6         4247     0.04247
## 7           18     0.00018
```

```
## 8           18     0.00018
## 9            2     0.00002
```

The diabetes dataset has 100,000 rows and no missing values. The target is binary, so it is valid for AUC/ROC analysis. The positive class is rare: about 8.5% of observations have `diabetes = 1`.

```
prop.table(table(diabetes_data$data$diabetes))
```

```
##
##     0     1
## 0.915 0.085
```

```
aggregate(
  diabetes_data$data[c("age", "bmi", "HbA1c_level", "blood_glucose_level")],
  by = list(diabetes = diabetes_data$data$diabetes),
  FUN = mean
)
```

```
##   diabetes      age      bmi HbA1c_level blood_glucose_level
## 1        0 40.11519 26.88716    5.396761            132.8525
## 2        1 60.94659 31.98838    6.934953            194.0947
```

The class means already show a clear pattern: observations with diabetes are older on average and have higher BMI, HbA1c level, and blood glucose level. This does not prove causality, but it identifies variables that are useful to inspect more closely.

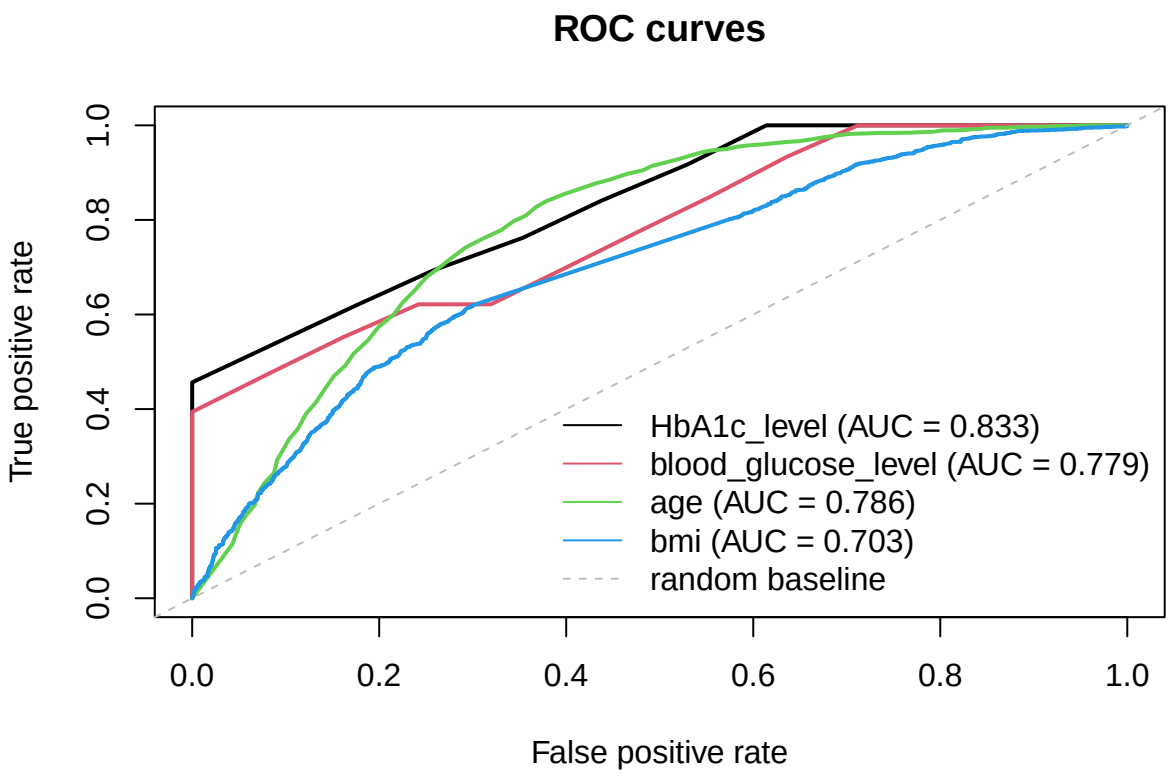
```
set.seed(7)
diabetes_sample <- rbind(
  diabetes_data$data[diabetes_data$data$diabetes == 0, ][sample(sum(diabetes_data$data$diabetes == 0), 1
  diabetes_data$data[diabetes_data$data$diabetes == 1, ][sample(sum(diabetes_data$data$diabetes == 1), 1
)

diabetes_features <- c("age", "bmi", "HbA1c_level", "blood_glucose_level", "hypertension", "heart_diseas
diabetes_metrics <- attribute_metrics(diabetes_sample[c(diabetes_features, "diabetes")], target = "diab
diabetes_metrics[diabetes_metrics$metric == "auc", ][order(diabetes_metrics[diabetes_metrics$metric ==
```

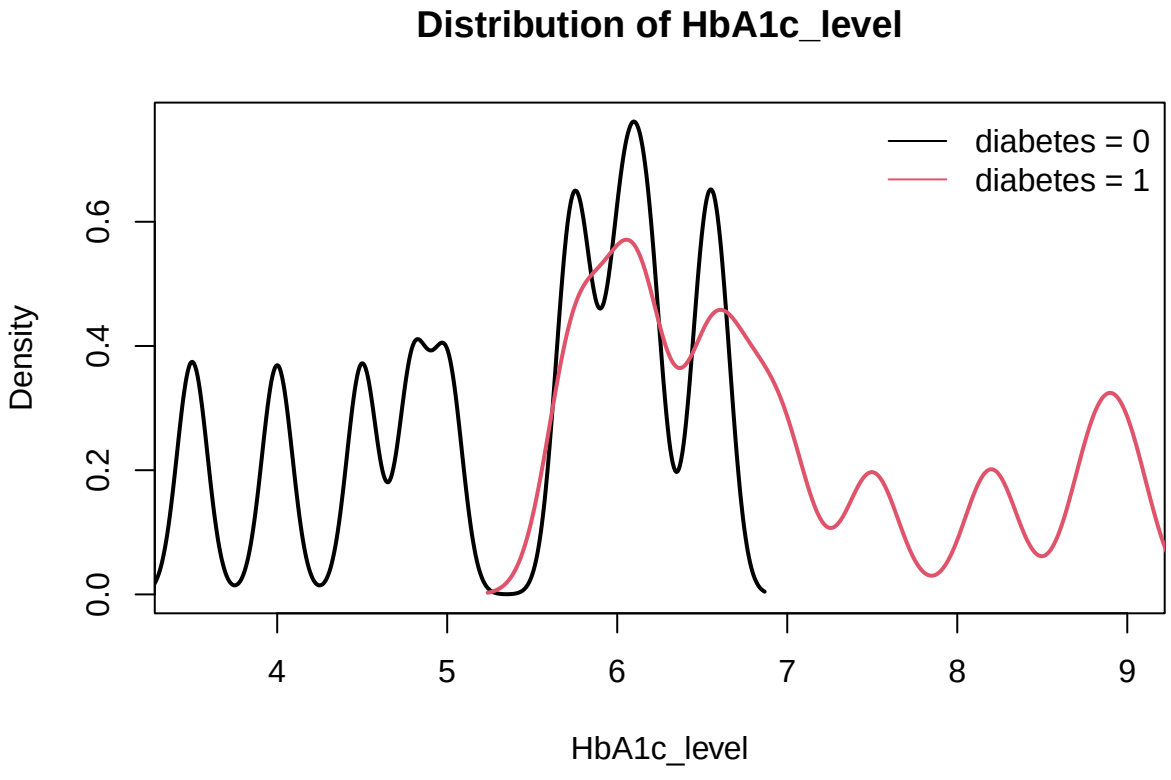
```
##              attribute metric     value
## 6          HbA1c_level    auc 0.8332381
## 2                  age    auc 0.7855661
## 8  blood_glucose_level    auc 0.7788721
## 4                  bmi    auc 0.7027521
## 10        hypertension    auc 0.5917500
## 12       heart_disease    auc 0.5637500
```

The package AUC is implemented with pairwise comparisons for educational clarity. To keep this example fast, we use a reproducible balanced sample. In this sample, `HbA1c_level` and `blood_glucose_level` are the strongest single numerical separators, followed by `age` and `bmi`. The binary medical-history variables are still informative, but their AUC values are lower as single-variable ranking scores.

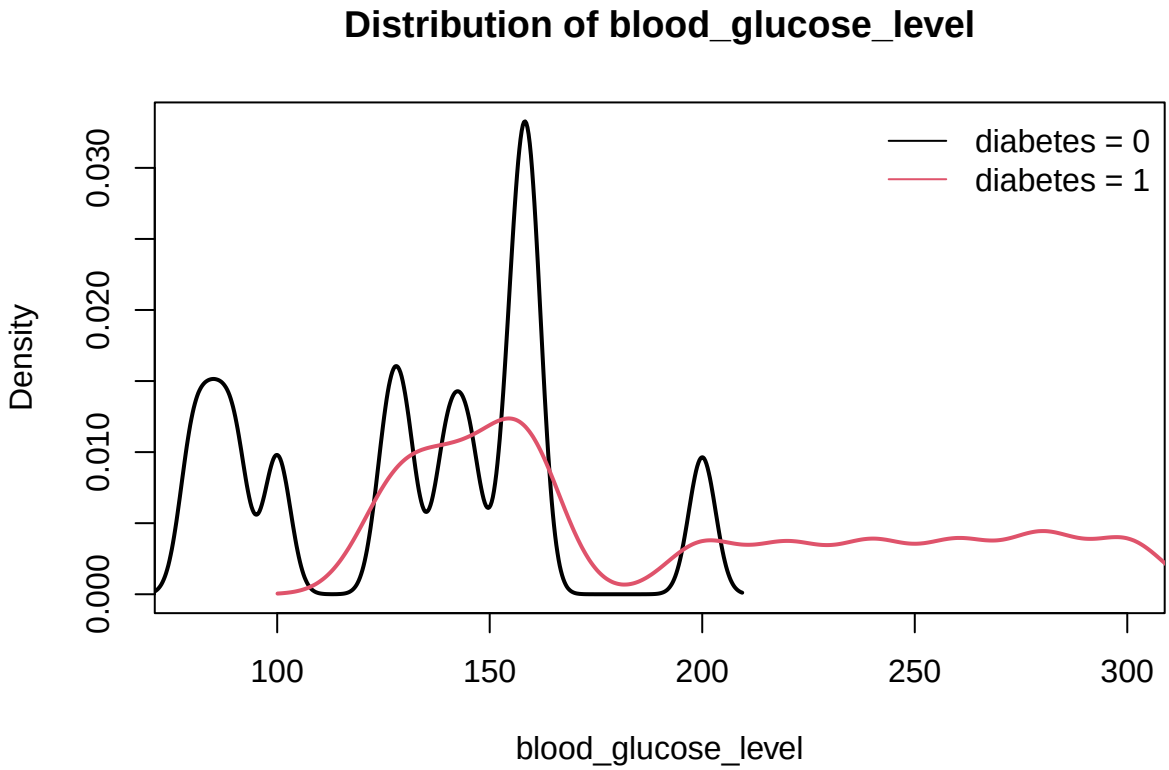
```
plot_roc_curves(diabetes_sample, target = "diabetes", columns = c("HbA1c_level", "blood_glucose_level",
```




```
plot_variable_distribution(diabetes_data$data, variable = "HbA1c_level", target = "diabetes", by_class =
```




```
plot_variable_distribution(diabetes_data$data, variable = "blood_glucose_level", target = "diabetes", by
```



The ROC curves confirm the metric table: HbA1c and blood glucose are the most useful individual numerical variables in this quick analysis. The distribution plots explain why: the positive class is shifted toward higher HbA1c and glucose values. A practical preprocessing step for a later model would be to keep these variables, keep age and BMI as supporting predictors, and treat the class imbalance explicitly.